

Vulnerability: SQL Injection in Login (Authentication Bypass)

CWE: CWE-89 - Improper Neutralization of Special Elements used in an SQL Command

OWASP: A03:2021 - Injection

CVSS: 9.8 (Critical)

Location: routes/login.ts

Component: Backend API - Login endpoint (POST /rest/user/login)

Description

The login endpoint constructs its SQL query by directly interpolating user-supplied input (req.body.email and the hashed req.body.password) into the query string using JavaScript template literals. Because the input is not parameterized, an attacker can inject arbitrary SQL syntax to manipulate the query's logic and bypass authentication entirely.

Vulnerable code:

models.sequelize.query(`SELECT \* FROM Users WHERE email = '\${req.body.email} || ''' AND password = '\${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })

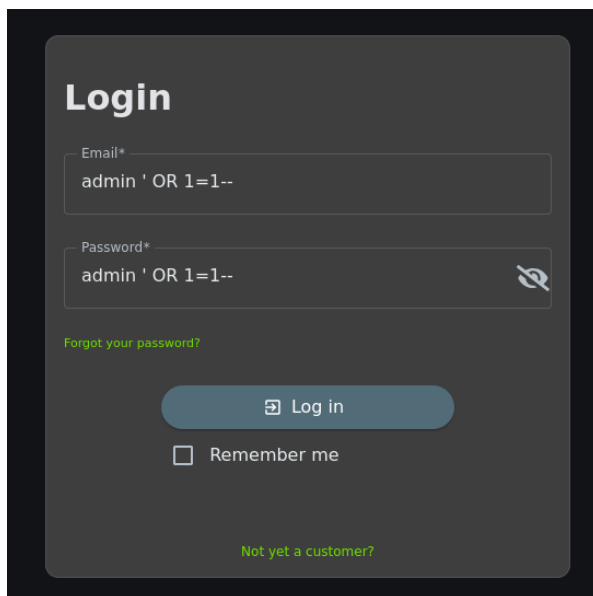
```
1 return (req: Request, res: Response, next: NextFunction) => {
2   verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
3   models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email} || ''' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })
4   const user = utils.queryResultToJson(authenticatedUser)
5   if (user.data?.id && user.data.totpSecret !== '') {
6     res.status(401).json({
7       status: 'totp_token_required',
8       data: {
9         tmpToken: security.authorize({
10           userId: user.data.id,
11           type: 'password_valid_needs_second_factor_token'
12         })
13       }
14     })
15   } else if (user.data?.id) {
16     afterLogin(user.data, res, next)
17   } else {
18     res.status(401).send(res.__(`Invalid email or password.`))
19   }
20 }.catch((error: Error) => {
21   next(error)
22 })
23 }
```

```
1 models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email} || ''' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })
```

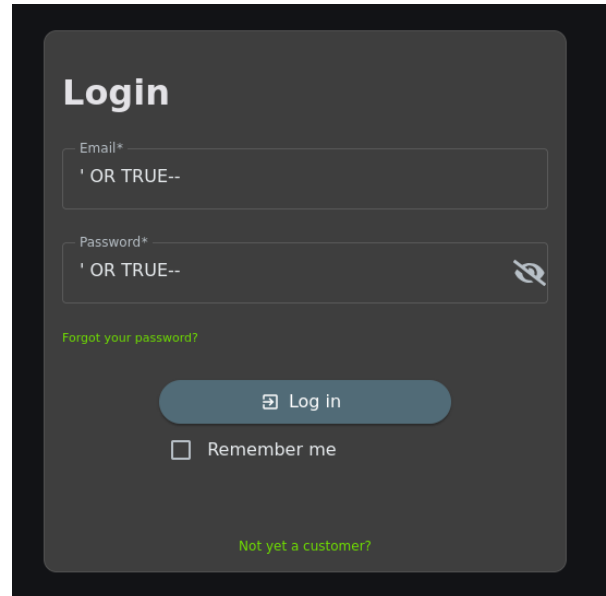
Exploit payload: ' OR TRUE --

admin' OR 1=1--

1. Navigate to <http://localhost:3000/#/login>
2. Enter ' OR TRUE-- or admin' OR 1=1-- in the email field.
3. Enter any value in the password field.
4. Submit the login form.



The screenshot shows a login form titled "Login" on a dark background. It has two input fields: "Email*" and "Password*". Both fields contain the text "admin ' OR 1=1--". Below the password field is a "Forgot your password?" link. At the bottom, there is a "Log in" button with a right-pointing arrow icon, a "Remember me" checkbox, and a "Not yet a customer?" link.



The screenshot shows the same login form as the previous one, but with the text "' OR TRUE--" entered in both the "Email*" and "Password*" fields. All other elements, including the "Log in" button, "Remember me" checkbox, and "Not yet a customer?" link, are identical.

Resulting query executed by the database:

```
SELECT * FROM Users WHERE email = " OR TRUE--" AND password = '<hash>' AND deletedAt IS NULL
```

The Vulnerability:

' OR TRUE --

The application builds the SQL query by directly concatenating user input. The payload ' OR TRUE-- manipulates the query logic,

Part of payload	Effect on the query
'	Closes the email string literal early
OR TRUE	Makes the entire WHERE clause always evaluate to true
--	Comments on the rest of the query, including the password check

The database ends up returning the first user to the table (the admin) and lets the attacker in without checking a password.

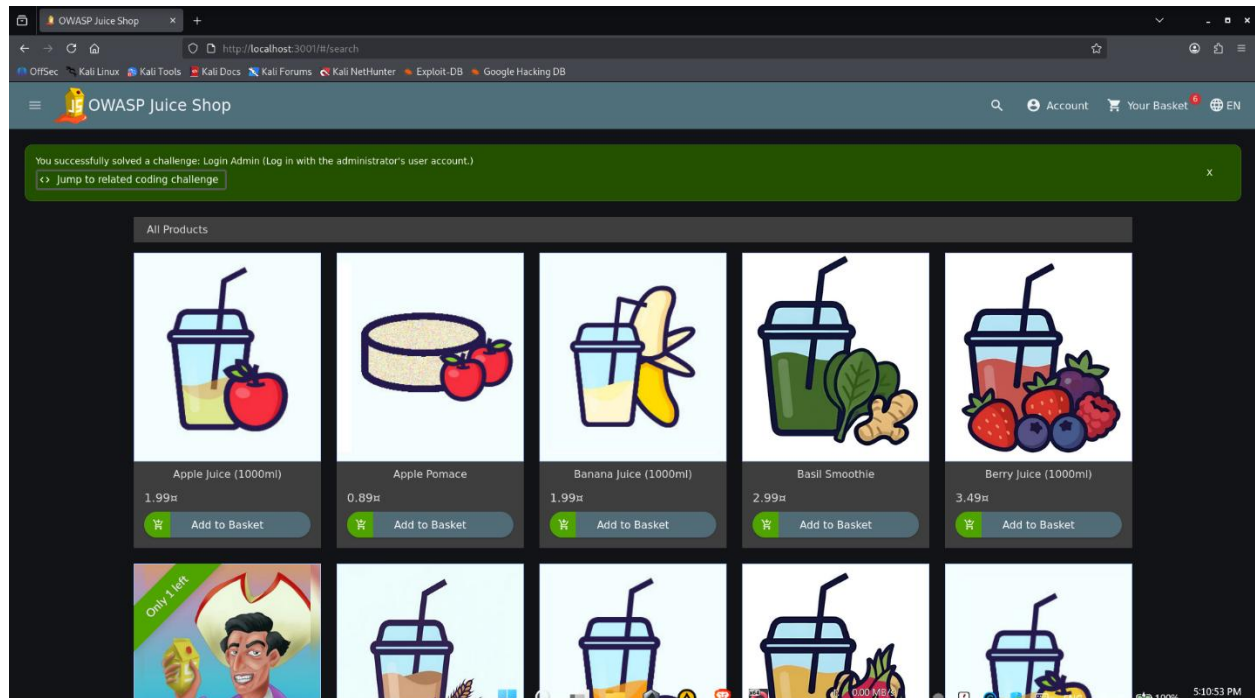
admin' OR 1=1--

This payload targets the admin account specifically: it breaks out with ', adds OR 1=1 always true, then -- comments out the password check.

The database returns the first matching row, the administrator user. The application then issues a valid JWT for that user, granting the attacker full admin access without ever knowing a password.

Evidence - Before Fix

- Login form with ' OR TRU-- or admin' OR 1=1-- entered in the email field and any value in the password field.
- Successful authentication as the administrator account - admin@juice-sh.op, confirming the bypass worked.



SAST Detection - Before Fix

Tool: Semgrep

Rule: sequelize-template-literal-injection

Run against the unmodified routes/login.ts, Semgrep flags the interpolated query. Capture the finding count and the specific rule ID and line number flagged.

```
kali@kali: ~/juice-shop-lab/juice-shop
$ cd ~/juice-shop-lab/juice-shop
semgrep scan --config .semgrep-rules/ \
  --metrics=off \
  routes/login.ts \
  --json --output=semgrep-login-before.jsonp
$ node build/app.js
li kali 1892 Sep 24 16:33 build/app.js
Semgrep CLI
$ npm start
Loading rules ...
Scanning 1 file (only git-tracked) with 1 Code rule:
> node build/app.js
CODE RULES
Scanning 1 file: e.js version v24.19.0 (SUCCESS)
info: Detected OS linux (SUCCESS)
SUPPLY CHAIN RULES
info: Configuration default validated (SUCCESS)
No rules to run: 21 of 21 are initialized (SUCCESS)
info: All dependencies in ./package.json are satisfied (SUCCESS)
info: Required file server.js is present (SUCCESS)
PROGRESS
info: Required file index.html is present (SUCCESS)
info: Required file styles.css is present (SUCCESS)
100% 0:00:00
info: Required file main.js is present (SUCCESS)
info: Required file matching /^hacking-instructor-.*\.js$/ is present (SUCCESS)
info: Port 3000 is available (SUCCESS)
https://www.alchemy.com/ is reachable (WARNING)
variable ALCHEMY_API_KEY is not present (WARNING)
"Honey Pot" challenge will not work as intended with
Injection" challenge will not work as intended with
Manipulation" challenge will not work as intended without
Ran 1 rule on 1 file: 1 finding. challenge will not work as intended
```

Secure Coding Fix

The correct remediation is to use **parameterized queries**. In Sequelize, this is achieved by replacing interpolated values with (?) placeholders and passing the values via the replacements array. The database driver then binds each value as data, not SQL syntax.

```
models.sequelize.query( 'SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL', {replacements: [req.body.email || '', security.hash(req.body.password || '')], model: UserModel, plain: true})
```

```
1 return (req: Request, res: Response, next: NextFunction) => {
2   verifyPreloginChallenges(req) // vuln-code-snippet hide-line
3   models.sequelize.query( 'SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL', {replacements: [req.body.email || '', security.hash(req.body.password || '')], model: UserModel, plain: true})
4   .then(authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginDinChallenge
5     const user = utils.anyResultToJson(authenticatedUser)
6     if (user.data?.id && user.data.totpSecret !== '') {
7       res.status(401).json({
8         status: 'totp_token_required',
9         data: {
10           token: security.authorize({
11             userId: user.data.id,
12             type: 'password_valid_needs_second_factor_token'
13           })
14         }
15       })
16     } else if (user.data?.id) {
17       afterLogin(user.data, res, next)
18     } else {
19       res.status(401).send(res.__( 'Invalid email or password.'))
20     }
21   }).catch((error: Error) => {
22     next(error)
23   })
24 }
```

```
1 models.sequelize.query( 'SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL', {replacements: [req.body.email || '', security.hash(req.body.password || '')], model: UserModel, plain: true})
```

Why this works: With parameterization, the payload ' OR TRUE-- is sent to the database as a literal string value for the email column. The database looks for a user whose email is literally ' OR TRUE-- which does not exist. No user is returned, and authentication fails.

Evidence - After Fix

Login

Invalid email or password.

Email*

' OR TRUE--

Password*

' OR TRUE--

Forgot your password?

Log in

Remember me

Not yet a customer?

Login

Invalid email or password.

Email*

admin' OR 1=1--

Password*

admin' OR 1=1--

Forgot your password?

Log in

Remember me

Not yet a customer?

OWASP Juice Shop

Login

Invalid email or password.

Email*

' OR TRUE--

Password*

' OR TRUE--

Forgot your password?

Log in

Remember me

or

Log in with Google

Not yet a customer?

Network

styles.css

whoami

whoami?fields=email

login

1 Invalid email or password.

Re-running the same scan on the patched login.ts shows the SQL injection finding is gone. Capture the after-fix finding count.

```

kali@kali:~/juice-shop-lab/juice-shop$ cd ~/juice-shop-lab/juice-shop
kali@kali:~/juice-shop-lab/juice-shop$ semgrep scan \
--config .semgrep-rules/ \
--metrics=off \
--json --output=semgrep-login-before.json
-rw-rw-r-- 1 kali kali 1892 Sep 24 16:33 build/app.js
Semgrep CLI | ~/juice-shop-lab/juice-shop

Loading rules ... 0 start
Scanning 1 file (only git-tracked) with 1 Code rule:

CODE RULES
Scanning 1 file.
INFO: Detected CPU x64
SUPPLY CHAIN RULES
default validated
INFO: Entity models 21 of 21 are initialized
No rules to run.
INFO: Required file server.js is present
INFO: Required file index.html is present
PROGRESS
INFO: Required file styles.css is present
INFO: Required file polyfills.js is present
100% 0:00:00
INFO: Required file matching /*hacking-instructor-.*\.js$/ is present
INFO: Port 3000 is available
INFO: Domain https://www.alchemy.com/ is reachable
Warning: Environment variable ALCHEMY_API_KEY is not present
Scan Summary
Scan completed successfully.
Findings: 0 (0 blocking)
Rules run: 1
Targets scanned: 1
Parsed lines: ~100.0%
No ignore information available
Ran 1 rule on 1 file: 0 findings.
If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/

```

Before / After Summary

Metric	Before Fix	After Fix
Semgrep finding count on login.ts	1	0
Exploit payload ' OR TRUE--	Admin login succeeds	Login rejected
SQL query construction	String interpolation	Parameterized (? + replacements)
Root cause present	Yes	No
CWE-89 compliant	No	Yes